

SECTION 23 — TEST DATA MANAGEMENT (Refined)

Topics Covered

- 23.1-23.14 (14 headers)

First pass — 2 parallel batch subagents

23.1 Static Test Data — Refined

Theory: Pre-defined, fixed data stored in JSON/CSV/SQL fixtures or config files, reused across runs. Determinism, easy version control, fast setup for stable reference data (countries, roles, product catalogs). Drawbacks: staleness, coupling, collisions in parallel runs. Best for non-target inputs (data the test reads but doesn't mutate), boundary values, negative cases requiring exact expectations. Refresh periodically; isolate mutable copies per run. Never hardcode credentials — load via environment profiles.

Enterprise Relevance: Static fixtures are the fastest setup (no creation cost) but the top collision source at scale.

Deloitte rule: static for reference data, dynamic for entities under test; mutable copy per run when a test must modify fixture data.

```
{ "username": "static_admin", "role": "admin" }
```

High-Stakes Scenario: 8 workers share `users.csv` with 10 fixed emails — duplicate-key failures + cross-test overwrites. Triage: reference data stays static (countries), entities become dynamic (users/orders); per-run copies for mutation; version fixtures with code.

Anti-Patterns: Mutable entities in static files; shared credential pools; hardcoded prod data; fixture drift (code changed, fixture didn't).

Qs: Static vs dynamic — what belongs where? How do you prevent static-fixture collisions?

23.2 Dynamic Test Data — Refined

Theory: Generated at runtime via API, DB seeding, libraries (Faker), or synthetic-data services instead of static files. Fresh, isolated, parallel-safe runs and edge-case simulation on demand. SDET best practice is Just-In-Time creation: each test creates and authenticates its own data via setup hooks, then deletes it. Avoids stale-data flakiness and grey-box DB inserts that bypass real business logic. Cost: extra setup time + tooling, offset by reliable parallelization and higher coverage.

Enterprise Relevance: JIT is what makes parallelism safe — no shared rows to collide on. Setup cost (200ms API create) pays back in zero collision flakes + no nightly cleanup jobs.

```
const email = `user_${Date.now()}_${testInfo.testId}@test.com`;
const user = await api.createUser({ email }); // own data
// ... test ...
await api.deleteUser(user.id);                // own cleanup
```

High-Stakes Scenario: Grey-box DB inserts bypass validation → tests pass on invalid data prod rejects (false confidence). Triage: create through API (exercises real validation), reserve direct DB for states APIs cannot produce (legacy, edge constraints) with explicit justification.

Anti-Patterns: Direct DB inserts for everything (bypasses logic); no cleanup (storage bloat + order-dependence); shared "test user" across suite.

Qs: JIT vs static — cost/benefit? When is direct DB seeding justified vs API?

23.3 Test Data Factories — Refined

Theory: Factories centralize object creation using Factory/Builder patterns with sensible defaults and explicit overrides. Instead of scattered `new User(...)` setup, tests declare intent: `TenantFactory()`, `AdminUserFactory(tenant)`. Libraries: FactoryBot, factory_bot, Fishery. Factories resolve relationships via SubFactory, integrate with fixtures for DB/test-client context, and keep tests readable by including only expectation-relevant fields. Avoid mystery guests (hidden defaults the test depends on but never shows) and module-level shared instances; call factories inside each test.

Enterprise Relevance: Factories cut test setup from 30 lines to 3 and make intent explicit for reviewers. Deloitte pattern: `UserFactory.create({role: 'admin'})` — overrides only what matters, defaults cover the rest, relationships auto-wired.

```
admin = AdminUserFactory(tenant=tenant)      # intent: admin in this tenant
epd = EPDFactory(tenant=tenant, gwp=999999)  # override only GWP
```

High-Stakes Scenario: 50 tests construct users inline with 20 fields each — a required-field change breaks all 50.

Triage: factory defaults updated once; tests specifying only relevant fields unaffected; mystery-guest failures (hidden default changed meaning) fixed by explicit overrides in affected tests.

Anti-Patterns: Module-level shared instances; hidden defaults tests implicitly rely on; god-factories with 40 params; factories performing UI actions (slow).

Qs: Factory vs Object Mother vs Builder? What is a mystery guest and how do you kill it?

23.4 Unique Data Generation — Refined

Theory: Unique data prevents collisions, duplicate-key failures, and cross-test interference in parallel CI. Techniques: UUIDv4/v7, ULID, timestamps, counters, worker-scoped prefixes (`api-test-<runId>-<workerIndex>`). UUIDv4 for random IDs, UUIDv7 when time-ordered sorting matters, deterministic UUIDv5 only for reproducible seeds. Combine timestamp + random suffix for readable emails/names. Always capture authoritative IDs returned by creation APIs for later cleanup, rather than relying on names alone for identity.

Enterprise Relevance: Uniqueness is non-negotiable under parallelism — the cheapest flake fix with highest ROI.

Deloitte standard: every created entity carries run/worker/test identity; cleanup keys off authoritative IDs.

```
import { randomUUID } from "crypto";
const email = `test_${randomUUID()}@example.com`;
const res = await api.post('/users', { data: { email } });
const id = res.id; // authoritative – use for verify + cleanup, not email lookup
```

High-Stakes Scenario: Timestamp-only uniqueness (`Date.now()`) collides when 8 workers create in the same

millisecond. Triage: timestamp + random/UUID hybrid; workerIndex prefix for debuggability (`qa-run412-w3-...` tells

you exactly which worker/run created it).

Anti-Patterns: Timestamps alone; counters without worker scope; names as identity (renames break cleanup); UUIDv5 "random" misuse (deterministic != unique).

Qs: UUIDv4 vs v7 vs ULID — when each? Why capture IDs instead of querying by name?

23.5 Test User Management — Refined

Theory: Provision isolated users per test/role with least-privilege RBAC coverage: admin, viewer, support, customer. Prefer programmatic creation via management API with `test:true` flag, generating OTP/magic-link for auth-flow validation, rather than sharing static credential pools (locking + flakiness). Pass identity via headers or tokens, scope users to tenant, pre-build role fixtures for speed. Clean up users post-run; never hardcode secrets — inject via environment variables or secret manager across staging and CI.

Enterprise Relevance: Users are the most contended test resource (sessions, rate limits, audit trails). Per-test users + RBAC matrix is how Deloitte proves authorization coverage (horizontal + vertical) without flake.

```
const user = await createTestUser({ role: "admin", tenantId, test: true });
await loginAs(user, authHeaders(user.id));
// role fixtures: admin/viewer/support/customer pre-built, cloned per test
```

High-Stakes Scenario: Shared `qa-admin` locked after 5 failed logins (account lockout policy) — entire suite red, security team alerted. Triage: per-test users (no shared lockout surface), distinct passwords (no credential-stuffing trip), `test:true` flag excludes from fraud monitoring, cleanup deletes users (no accumulation).

Anti-Patterns: Shared credential pool; hardcoded passwords; admin for every test (no RBAC signal); no cleanup (10k orphan users break other tests' queries).

Qs: How do you cover the RBAC matrix without N×M test explosion? How do you test OTP/magic-link flows?

23.6 Data Isolation — Refined

Theory: Data isolation ensures a test yields the same result alone, in sequence, or in parallel by eliminating shared mutable state. Strategies: fresh WebDriver/browser context per test, separate DB schema/container per worker, tenant-per-run partitioning, transaction rollback, unique namespacing with `workerIndex`. SauceLabs model contrasts grab-and-hope, static fixture, dynamic fixture with preferred Just-In-Time: each test owns all data it touches.

Complement with `@Isolated` guards for global resources, resource locks, randomized ordering to expose hidden dependencies.

Enterprise Relevance: Isolation is the prerequisite for parallelism, retries, and sharding combined. A test passing alone but failing in suite is, by definition, an isolation failure — treat the test as buggy, not the suite as flaky.

```
test.use({ storageState: { cookies: [], origins: [] } }); // clean slate
// + tenant-per-run: test_tenant_${runId}_w${workerIndex}
// + rollback: afterEach → ROLLBACK (no residue)
// + randomized order in CI to expose hidden deps
```

High-Stakes Scenario: Suite green in fixed order, red shuffled — 12 hidden dependencies (test 45 reads test 12's row). Triage: randomized order job (finds all), per-test ownership fix, `@Isolated` only for true globals (feature flags), dependency graph documented and eliminated.

Anti-Patterns: Order-dependent suites ("run login first"); shared tenant across workers; no cleanup ("someone else's

test cleans"); fixed order masking dependencies.

Qs: How do you prove isolation? What are the 4 isolation strategies and when each?

23.7 Data Cleanup — Refined

Theory: Cleanup removes automation-created records to prevent pollution, flakiness, and storage bloat. Reliable pattern is ownership-based teardown: register delete-by-ID immediately after each create, delete in reverse dependency order (child before parent), continue-on-error while reporting failures. Since mid-run crashes skip teardown, add startup sweepers that delete only tagged, expired records in test tenants (dry-run limits + audit logs). Prefer `DELETE WHERE run_id=?` over `TRUNCATE`; keep teardown assertion-free; version cleanup SQL/API scripts in Git.

Enterprise Relevance: Without cleanup, staging accumulates 100k orphan rows → queries slow, uniques collide, other teams' tests break on your data. Cleanup is a first-class suite component with owner + monitoring, not an afterthought.

```
-- ownership-based, reverse order, assertion-free
DELETE FROM order_items WHERE order_id IN (SELECT id FROM orders WHERE run_id = 'run412');
DELETE FROM orders WHERE run_id = 'run412';
DELETE FROM users WHERE run_id = 'run412';
-- sweeper (startup): expired tagged orphans only, LIMIT + audit
DELETE FROM users WHERE run_id LIKE 'qa-%' AND created_at < NOW() - INTERVAL '24 hours' LIMIT 10
```

High-Stakes Scenario: `TRUNCATE users` in parallel run wipes other workers' + manual QA's data — incident. Triage: `WHERE run_id` scoping mandatory (lint for bare `TRUNCATE/DELETE-without-WHERE` in tests), per-run `run_id`, sweepers tagged+expired only, dry-run mode for sweeper changes.

Anti-Patterns: `TRUNCATE` in parallel; no cleanup; assertions in teardown (masks original failure); cleanup of untagged data (deletes others'); unversioned manual cleanup scripts.

Qs: Ownership-based vs sweeper — when each? Why is `TRUNCATE` banned in parallel suites?

23.8 Seed Data — Refined

Theory: Versioned seed scripts create deterministic baseline data before suite runs. Run migrations then `prisma db seed` in `globalSetup`, or restore SQL snapshot (`pg_dump/psql`) for speed. Seed only reference data — categories, flags, admin — idempotently (accept 409 on re-run). Per-test mutable data belongs in fixtures, not global seed. Store run metadata via env or JSON for tests to consume; clean in `globalTeardown`. Never seed production; target disposable staging database that CI can reset reliably.

Enterprise Relevance: Seeding is suite startup — slow/flaky seeding delays every run. Snapshot restore (seconds) beats migration+seed (minutes) at scale; idempotency allows rerun without manual DB wipe.

```
// global-setup.ts — deterministic baseline, idempotent
import { execSync } from 'child_process';
export default async function globalSetup() {
  execSync('npx prisma migrate deploy', { stdio: 'inherit' });
  execSync('npx prisma db seed', { stdio: 'inherit' }); // upserts only, 409-safe
}
```

High-Stakes Scenario: Seed fails halfway — half-seeded DB, 500 tests fail with confusing errors. Triage: seeding in transaction (all-or-nothing) or snapshot restore (atomic file swap); seed verification query (counts per table) before

suite starts; `globalTeardown` cleanup; seed version pinned with app migration version.

Anti-Patterns: Mutable test entities in global seed; non-idempotent seed (fails on rerun); seeding prod; seed data without versioning (drift vs app).

Qs: Seed vs fixture vs factory — what lives where? How do you make seeding atomic and fast?

23.9 API-Based Data Setup — Refined

Theory: Prefer Playwright `APIRequestContext` over UI clicks for setup: faster (ms vs seconds), less flaky, and exercises real backend validation. Use `request` fixture or `playwright.request.newContext({ baseURL })` with auth token to POST users/products/issues, then drive UI assertions against seeded IDs. Wrap creation in fixtures tracking IDs with DELETE in reverse dependency order after `use()`. Login once via API, persist `storageState` for browser contexts. Always `dispose()` standalone contexts; assert `res.ok()` to fail fast on seeding errors. Enterprise Relevance: API seeding is the single biggest suite-speed lever (14s UI signup → 200ms API create; Sec 34 case: -47min/200 tests). It also validates backend as a side effect — setup failures are product bugs, not test bugs.

```
const api = await request.newContext({ baseURL, extraHTTPHeaders: { Authorization: `Bearer ${pro
const res = await api.post('/api/products', { data: { name: 'Seeded Widget', price: 1999 } });
expect(res.ok()).toBeTruthy();
const { id } = await res.json();
await page.goto(`/products/${id}`); // UI asserts on API-seeded reality
await api.dispose();
```

High-Stakes Scenario: UI-setup (14s × 200 tests = 47min) dominates suite; team blames "slow tests". Triage: API-seed everything creatable (200ms), UI login replaced by `storageState`, per-test IDs tracked for cleanup; suite 60 → 13min with zero test-logic changes.

Anti-Patterns: UI clicks for setup data; no `res.ok()` assert (seed failure → confusing downstream error); undisposed contexts (socket leaks); seeding via UI then asserting same UI (circular).

Qs: What setup must stay UI vs move to API? How do you clean API-seeded data?

23.10 Database-Based Setup — Refined

Theory: Use direct DB access when APIs cannot create required states: legacy data, edge constraints (e.g., backdated rows, invalid-for-API-but-possible-in-DB states), or performance (bulk seed 10k rows). Provision worker-scoped `pg.Pool`, run migrations in `globalSetup`, expose `seedTask` / `findTaskByTitle` fixtures. Classic patterns: seed → UI verifies rendering, UI → DB verifies persistence, API → DB verifies writes, plus negative unique-constraint checks. Docker Postgres on dedicated port away from dev DB. Query explicitly by ID/title (never "latest row"); cleanup scoped rows. Raw SQL for setup verification, not business-logic bypass.

Enterprise Relevance: DB seeding is the escape hatch for states the API forbids — necessary for negative/legacy coverage, dangerous when overused (bypasses validation = false confidence). Every direct-DB seed needs a justification comment.

```
export async function seedTask(db: Pool, title: string) {
  const r = await db.query('INSERT INTO tasks(title) VALUES($1) RETURNING *', [title]);
  return r.rows[0]; // authoritative row, explicit ID downstream
}
```

High-Stakes Scenario: Tests insert via DB bypassing `NOT NULL` app-level checks → suite green, prod rejects real payloads (gap). Triage: default API seeding; DB only with `// DB-ONLY: backdated 2020, API forbids past dates` justification; negative tests asserting constraint violations explicitly.

Anti-Patterns: DB for everything (validation bypass); "latest row" queries (order flake); shared dev DB (collisions); no cleanup (100k rows).

Qs: API vs DB seeding decision rule? How do you prevent validation-bypass blindness?

23.11 Parallel-Safe Data — Refined

Theory: Playwright isolates `BrowserContext` per test but not backend rows — parallel workers collide on shared emails/names. Derive unique identifiers from `testInfo.testId`, `workerIndex` / `parallelIndex`, plus CI run ID: `qa-{runId}-w{workerIndex}-slug`. Provide per-worker accounts via worker-scoped fixture (`user-${parallelIndex}`) or per-worker database (`pw_{run}_{slot}`). Never `TRUNCATE` globally under `fullyParallel: true`; delete only `WHERE title LIKE e2e-{runId}-%`. Nightly orphan cleanup for cancelled runs (which skip teardown).

Enterprise Relevance: Parallel-safety is a data architecture property, not a test-code property. Every parallel flake traced to data gets fixed in the data layer (namespacing), not with retries.

```
dbUserName: [async ({}, use) => {
  const name = `user-${test.info().workerIndex}-run${process.env.RUN_ID}`;
  await createUserInTestDatabase(name); await use(name);
  await deleteUserFromTestDatabase(name);
}, { scope: 'worker' }]
```

High-Stakes Scenario: `fullyParallel` rollout: 40% failures, all duplicate-key/collision. Triage: run-scoped prefix on every created entity, per-worker DBs for heavy suites, `TRUNCATE` ban lint, collision dashboard (group failures by constraint name to find shared entities).

Anti-Patterns: Shared emails across workers; global `TRUNCATE`; no run scoping (last week's orphans collide); per-test DB per test (too slow — per-worker instead).

Qs: What are the 3 scoping levels (test/worker/run) and what lives at each?

23.12 Environment-Specific Data — Refined

Theory: Parameterize `baseURL`, credentials, retries, and seed endpoints per environment via `TEST_ENV` and per-env `.env.local` / `.env.staging` files loaded with `dotenv`. Map env to config: local uses localhost with 0 retries, staging/prod use HTTPS URLs with retries. Alternatively define Playwright `projects: [{name: 'staging', use: {baseURL}}, {name: 'production'}]`. Never hardcode URLs in `page.goto` — use relative paths against `baseURL`. Inject CI values as real env vars overriding files; isolate datasets per environment to prevent staging seeds leaking into production assertions.

Enterprise Relevance: Env mix-ups (staging test writes prod) are incidents, not flakes. Per-env datasets + URL allowlists + relative navigation form triple protection.

```
const ENV = process.env.TEST_ENV || 'local';
dotenv.config({ path: `.env.${ENV}` });
use: { baseURL: envConfig[ENV].baseURL }
// tests: await page.goto('/orders'); // relative – env-safe
```

High-Stakes Scenario: Staging seed job runs against prod (wrong `.env` loaded) — 10k test rows in prod DB. Triage: CI env injection (files never contain prod), prod-write guard (read-only tags + approval), dataset isolation verified by pre-run check (assert env marker row), destructive ops require `--env` explicit flag.

Anti-Patterns: Hardcoded URLs; `.env.production` committed; absolute URLs in tests; shared dataset across envs; no env marker verification.

Qs: How do you make it impossible to run staging seeds against prod?

23.13 Sensitive Data and Secrets — Refined

Theory: Never hardcode passwords, tokens, or PII in specs. Load from git-ignored `.env` via `dotenv` at top of `playwright.config.ts`; validate required keys early. Commit only `.env.example` with dummy values. In CI inject secrets from GitHub Actions/Vault/Azure Key Vault, preferring managed identity and short-lived tokens. Reuse `storageState` from one setup project so credentials appear in one trace, not every test. Traces/videos capture typed secrets — prefer API login, avoid `console.log(process.env)`, rotate any committed secret immediately. Enterprise Relevance: Test code is a high-value secret-sprawl vector (dozens of contributors, traces uploaded to shared storage). One committed prod token = breach + audit finding + rotation across all envs.

```
// .gitignore: .env* playwright/.auth/
dotenv.config();
baseUrl: process.env.BASE_URL ?? 'http://localhost:3000'
// validate: if (!process.env.API_TOKEN) throw new Error('API_TOKEN missing');
```

High-Stakes Scenario: UI login typed prod password in 200 tests; traces uploaded to shared artifact store with readable keystrokes. Triage: API-login + `storageState` (secret appears once in setup trace, restricted access), mask password fields in reports, rotate credential, purge artifacts, Gitleaks + trace-secret scan in PR gate.

Anti-Patterns: Committed `.env`; `console.log(env)`; UI login per test with real creds; long-lived tokens; PII (real customer emails) as test data (GDPR).

Qs: API-login vs UI-login for secret hygiene? What do you do in the first 10min after committing a secret?

23.14 Test Data Anti-Patterns — Refined

Theory: The hall of shame: shared mutable staging data (two tests create `test@example.com` or edit the same invoice — order decides results, parallelism impossible); hardcoded emails/IDs (fail on second run); inter-test dependencies via module globals or `beforeAll` state; giant demo datasets obscuring intent; record-and-playback with embedded secrets; missing cleanup leaving orphans. Fix: generate unique values per test with factories/faker, create minimal data per test via API fixtures, assert on owned IDs, clean in `finally` /teardown with reverse-order deletes.

Enterprise Relevance: Every anti-pattern here has caused a real suite collapse. Deloitte code-review checklist encodes them as blocking comments: shared-mutable, hardcoded identity, missing cleanup, secrets in data.

```
// BAD: shared static user
const email = 'shared@test.com';
// GOOD: unique per test, owned lifecycle
const email = `user-${Date.now()}-${testInfo.testId}@example.test`;
const user = await api.createUser({ email });
try { /* test */ } finally { await api.deleteUser(user.id); }
```

High-Stakes Scenario: New-hire review checklist — scan any test-data file for: hardcoded emails, no cleanup, shared globals, `TRUNCATE`, secrets. Each finding links to this section's rule + fix pattern. Review time 30min, prevents months of flake debt.

Anti-Patterns: (this header IS the list — see above). Meta-anti: documenting without enforcing (add lint: no hardcoded `@example.com` without UUID, no `TRUNCATE` in tests).

Qs: Which 3 anti-patterns break parallelism? What does your review checklist enforce?